

GOLDFISH MODELS

Trading per-turn context for memory and throughput

Raghav Samavedam
CS 153 · Spring 2026

a goldfish with a notebook

(the regime)

Humans solve hard problems with ~4 items in working memory

Cowan (2001):

4 ± 1

items in active working memory at once.

Miller's earlier 7 ± 2 figure was for short-term capacity with chunking and rehearsal.

For immediate, untrained recall the number is closer to 4.

Where the actual work lives: long-term memory

Working memory is the bottleneck

Only ~4 items active at once. But humans solve enormous problems by retrieving from a vastly larger long-term store — the bulk of cognition is in retrieval, not in holding state.

Retrieval is precise, not fuzzy

A surgeon recalls a specific protocol step. A programmer recalls a specific function signature. We retrieve named, structured items — not vibes-similar associations.

And we structure memories as we learn

Studying a new domain, we deliberately organize: outlines, indexes, mnemonics, hierarchies. We don't just dump — we curate the store so future retrieval is cheap.

Goldfish = small working set (context window) + precise, structured long-term memory (the filesystem). Not fuzzy embedding search.

Long-context inference is bottlenecked by the KV cache

Per-token KV memory:

$$M_{kv} = 2 \cdot L \cdot n_{layer} \cdot n_{head} \cdot d_{head} \cdot b$$

(K + V) · sequence × per-token weight × bytes

Concrete example — Llama-3 70B, FP16:

$n_{layer} = 80$ $n_{head_kv} = 8$ $d_{head} = 128$
→ **~320 KB per token**

At $L = 32k$ tokens, batch = 16:

→ **164 GB** just for KV cache

The implication

KV cache grows linearly with both sequence length AND batch size.

Doubling context → halving max batch.

An 80 GB H100 holds ~7 concurrent 32k-token requests after weights and activations — not the hundreds you'd need to saturate the FLOPs.

The cache is the throttle, not compute.

Eliminating the per-turn cache → batch up by $\sim 15\times$

Setup: 80 GB H100, Llama-class 70B, 32k effective context.

Status quo

Per-turn context = 32,000 tokens

KV / request $\approx 32k \times 320 \text{ KB} \approx 10.2 \text{ GB}$

Free VRAM after weights & activations $\approx 70 \text{ GB}$

Max concurrent requests:

≈ 7

concurrent users / 80 GB GPU

Goldfish-2048

Per-turn context = 2,048 tokens

KV / request $\approx 2k \times 320 \text{ KB} \approx 0.66 \text{ GB}$

Same 70 GB of free VRAM available

Max concurrent requests:

≈ 105

concurrent users / 80 GB GPU

= $\sim 15\times$ throughput on long-horizon agentic workloads — for the cost of redesigning the agent harness, not the model.

The proposal: amnesia + a notebook

Every turn, the model sees only the last L tokens of conversation history.

1 Hard window cap

Per-turn context is truncated to $L \in \{128, 512, 1024, 2048\}$.

Nothing older than L tokens reaches the model. No retrieval. No summarization. Just truncation.

2 Filesystem persistence

The agent has a private directory between turns: notes.md, state files, files it grep'd, the repo it's editing.

Files survive what the context window evicts.

3 Light-touch protocol

The system prompt tells the model the regime exists — that the window is small, that the filesystem persists, that thinking should go to disk rather than into hidden tokens.

It does NOT prescribe a state file format.

Prior KV-eviction work answers the wrong question

All of these compress the cache by deciding which tokens to drop, based on a model-internal signal:

Method	What it keeps	Compute cost	What it implicitly assumes
H2O	Heavy-hitter tokens by attention	Online attention scoring	Attention magnitude $\approx$ semantic importance
SnapKV	Cluster centers per attention head	Pre-fill clustering	Recent context predicts what matters
NACL	Tokens by learned saliency	Auxiliary scorer per layer	Saliency transfers across tasks
InfiniPot	Continual-summary in-place	Cheap summary update	Summary captures the residue
Hashevict	Hash-bucketed approximation	Hash computation	Hash collisions $\approx$ semantic similarity
MorphKV	Morphing per-token retention	Learned policy	Single policy generalizes
RocketKV	Hierarchical token compression	Multi-stage scoring	Hierarchy aligns with usage

All of these use the model's own internal signal to decide what to keep — attention scores, learned scorers, hashes.

Goldfish skips that question entirely. We ask the model to write what it wants to remember, in shell. The decision is explicit and inspectable.

Setup: gpt-4o on SWE-bench Verified

Benchmark

SWE-bench Verified

- Real bugs in real Python repos (django, astropy, sympy)
- Model produces a unified-diff patch
- Official Docker scorer runs the held-out test suite
- Patch passes iff every FAIL_TO_PASS test now passes and every PASS_TO_PASS test still passes

L values swept:

$L \in \{ 512, 1024, 2048, \infty \text{ (native)} \}$

Per-turn output cap = L. Max turns = 20.

Why gpt-4o

Non-reasoning model. Hidden chain-of-thought does NOT compete with visible output for the same token budget.

When we tried gpt-5 with $L = \text{max_out}$, reasoning tokens consumed the entire per-turn budget. Every turn was 0 visible output, 100% LENGTH-CAP.

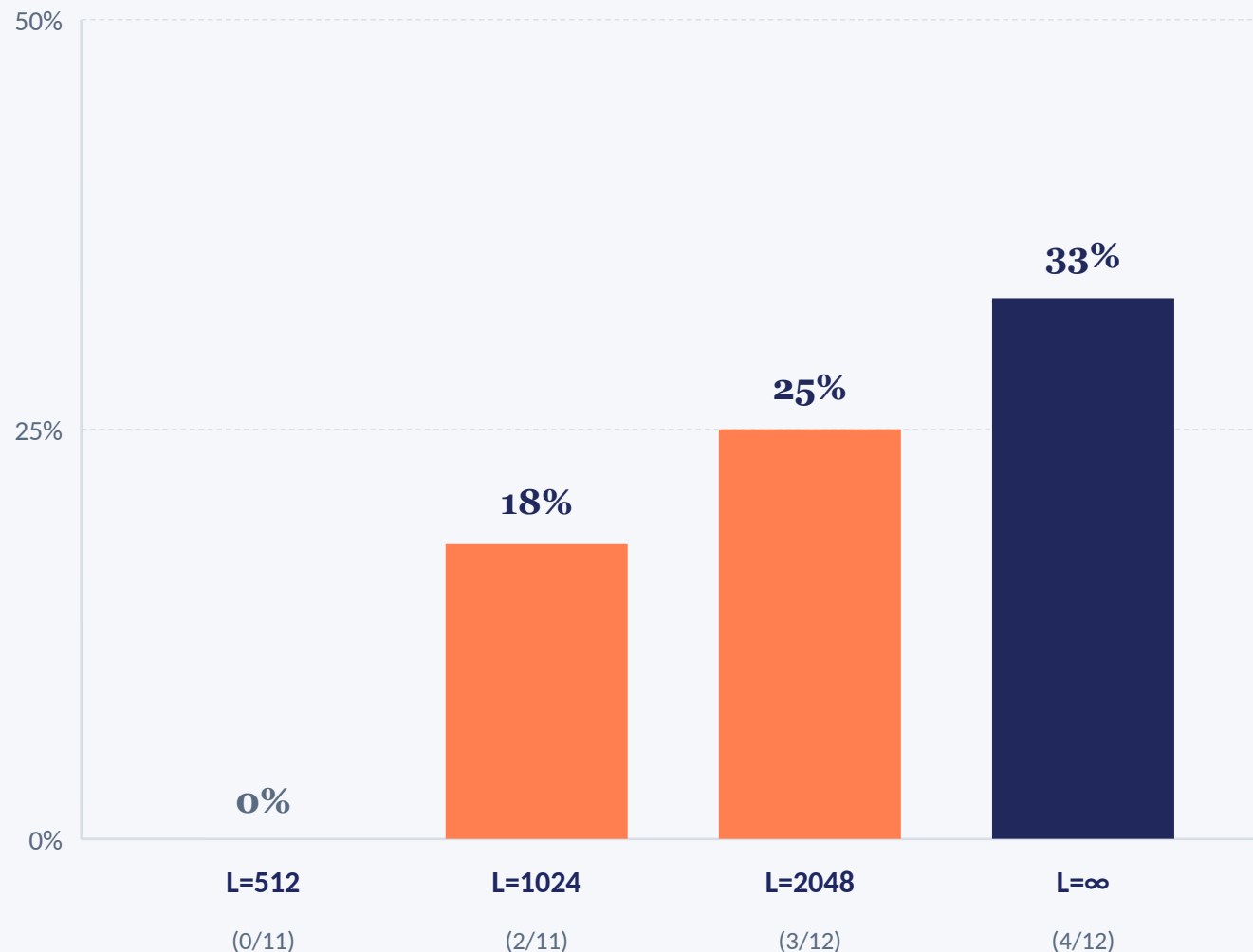
Goldfish is the wrong regime for opaque-reasoner architectures. The honest story.

Other choices that mattered:

- Auto-terminate when a valid unified-diff is exported
- Unique scorer run_id per call (avoids Docker collisions)

Solve rate climbs monotonically with L

Solve rate · tinstat variant · gpt-4o



Three things to notice

Hard floor at L=512

11 cells, zero solves. Below ~1k tokens the protocol cannot bootstrap — the model can't even fit its own notes back in.

L=2048 reaches 75% of native

25% vs 33% native. Most of the capability survives a 16× context cut.

The gap is task-specific

Three tasks solve at every $L \geq 1024$. One task is native-only. Most tasks don't solve at any L — gpt-4o ceiling.

What the model invents when L is small

Behaviors the prompt did NOT specify, observed in solving trajectories at $L \leq 2048$:

Comments as inline notes

Instead of writing notes.md as instructed, the model prefixes commands with # comments. Bash ignores them — but they appear TWICE in history.txt (as input + as echo), surviving truncation more robustly than a separate file.

Prose as scratch memory

On 'hypothesis turns' (typically T3), the model writes 2-3 sentences of diagnosis. That prose primes its own next several turns — visible output doing double-duty as both reasoning and persistent state.

sed → python3 fallback

Every solving trajectory tries sed first, fails on escaped characters, then falls back to python3 - <<PY for the real edit. Across 3 L values and 3 tasks. The model learned this tool preference on the fly.

Self-compression

Avg response length DECREASES as L decreases. At $L=1024$ the model writes ~68 tokens/turn; at $L=\infty$ it writes ~92. The constraint is a productivity feature, not just a memory limit.

From a real T2 response

($L=1024$, dj-11099, solved)

```
```bash
```

```
The relevant classes are located
in repo/django/contrib/auth/
validators.py. Let's inspect this
file to identify where the regex
needs to be updated.
sed -n '1,40p' \
 repo/django/contrib/
 auth/validators.py
```

```
```
```

The 5 comment lines carry the model's working memory across the turn boundary. Free storage.

Takeaways

Knowing what to keep is the hard problem.

Attention-score, hash, and learned-saliency methods compress the cache by guessing what mattered. Goldfish externalizes the decision: the model writes what it wants to remember.

Most native-context capability survives 16× compression.

$L=2048 \approx 75\%$ of $L=\infty$ solve rate, with a per-turn KV footprint $\sim 15\times$ smaller. The protocol is doing real work.

There is a floor.

Below $\sim 1k$ tokens the protocol cannot bootstrap; the act of reading your own notes consumes the window.

Reasoning models are wrong for this regime.

Hidden CoT competes with visible output. gpt-5 + $L=128$ emits zero visible tokens. Non-reasoning models cooperate; reasoning models do not.